

Tool、Skill 还是 Subagent：把失控 Prompt 拆回可评估 Agent 架构

Claude / Will Steuk; SSS & Codex

2026-07-07



视频作者/频道: Claude

发布日期: 2026-05-23

视频时长: 00:45:06

视频链接: <https://www.youtube.com/watch?v=mWvt0HlZM-I>

PDF 制作 Skill: youtube-render-pdf

目录

导读：本 PDF 的核心结论

这份笔记的核心结论是：当一个 agent 的 prompt、tools 和 subagents 全部堆在同一层时，系统会同时失去可评估性、可维护性和可解释性。Will Steuk 在视频中的拆解给出了一条更稳的工程路径：先用 evals 找到具体失败模式，再把长期知识下沉到 skills，把外部行动能力拆成 primitive tools、custom tools 或 MCP，把真正需要独立上下文和独立判断的任务升级为 subagent 或 Claude Managed Agents 的 callable agent。

全文围绕一个判断框架展开：**skill 管知识与流程，tool 管可执行动作，subagent 管独立认知任务**。三者的边界可以类比为“操作手册、工具箱、专门同事”：操作手册降低常驻 prompt 负担，工具箱提供稳定动作接口，专门同事负责需要隔离上下文、长期推进或独立诊断的工作。这个框架的风险在于边界判断会受项目规模、可观测性、工具运行成本和评估覆盖率影响；因此本文始终把架构建议回连到 F1、F2、R8 等 eval evidence，而不把架构偏好当作结论。

读者可按三层问题阅读：第一层理解 Stock Pilot 为什么从 400 行 prompt、12 个 tools 和 3 个 wrapper-style subagents 开始失控；第二层理解 eval-driven hill climbing 如何定位失败；第三层理解 CMA、skills、tools、subagents 如何分别吸收基础设施、知识、行动和独立认知这四类复杂度。

1 问题设定：一个 prompt 长到失控的库存 agent

1.1 结论先行：Stock Pilot 的问题已经是架构债

Stock Pilot 的核心问题可以先压缩成一句话：一个原本有效的库存管理 agent，因为业务能力持续追加，逐渐把长期规则、行动能力和独立判断都塞进同一个 orchestrator 周围，最终让系统 prompt、tool 列表和 subagent 边界一起膨胀。此时仅靠“把 prompt 写得更清楚”已经不足；工程上需要重新划分三类东西：什么知识按需加载为 skill，什么行动暴露为 tool，什么任务交给具有隔离上下文的 subagent。

本章主结论

当 agent 的能力增长来自持续“加东西”时，架构债通常先表现为三个信号：system prompt 变长、tool 数量变多、subagent 变成普通 tool wrapper。Stock Pilot 的退化属于知识、行动和独立认知三种责任被混放后的系统性后果。

Will Steuk 在开场用一个工程团队熟悉的场景建立问题：一个 agent 刚发布时工作得很好，于是几周后被要求增加 forecasting，再过几周又被要求增加 report writing、supplier selection、purchase order 等能力。每个单独需求都合理，累积后却让 agent 的上下文和执行面变得难以治理。这里的“长到失控”指向一种运行状态：模型每次处理任务时都要背负太多与当前任务无关的规则、工具和中间协议。

1.2 Stock Pilot 的业务边界：库存、预测、采购与报告

Stock Pilot 是一个面向中型户外装备零售商的 inventory management agent。画面给出的业务背景很具体：约 250 个 SKU、3 个仓库、12 个供应商，以及 90 天销售历史。它需要完成的是一组能落到操作层面的任务：监控低库存、预测需求、选择供应商、创建采购单、通知运营团队，以及

生成周报。

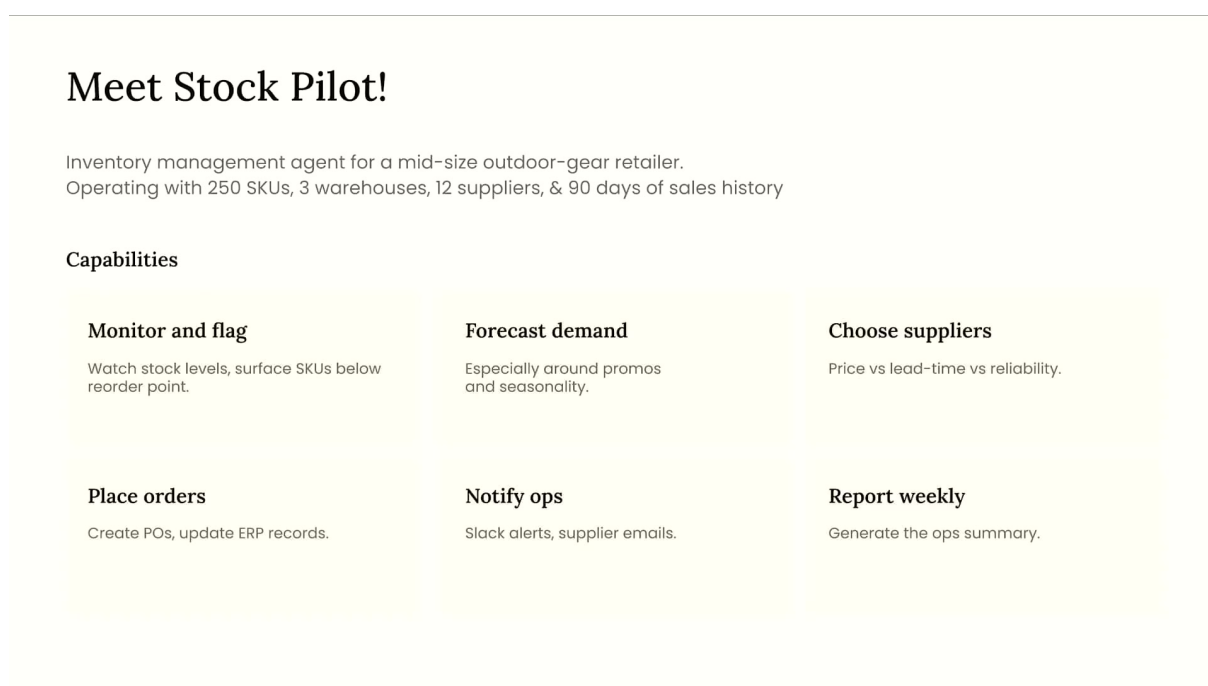


图 1: Stock Pilot 的业务能力范围：从库存监控到需求预测、供应商选择、采购单和周报生成。¹

这张图的重要性在于它说明 Stock Pilot 的复杂度来自真实业务链条。低库存检测偏向数据读取和阈值判断；需求预测需要处理促销、季节性和历史销售；供应商选择需要权衡价格、lead time 和可靠性；采购单与 ERP 更新属于外部系统动作；周报生成又涉及结构化写作和运营沟通。它们放在同一个 agent 中具有可成立的局部理由，风险来自后续实现方式：所有能力都被不断追加到同一套长期上下文和 tool surface 里。

三个核心对象的分工

tool 解决“能做什么动作”的问题，例如读取文件、运行代码、调用 API、写入订单。**skill** 解决“什么时候需要哪段知识”的问题，例如库存政策、预测流程、报告模板。**subagent** 解决“是否需要隔离上下文或并行认知”的问题，例如把 forecasting 放到独立 Claude 实例中，避免主 orchestrator 的上下文污染预测过程。

1.3 遗留架构画像：single orchestrator、400 行 prompt、12 个 tools

视频中的遗留架构由一个 single orchestrator 驱动。这个 orchestrator 运行在手写 Messages API while-loop 上，系统 prompt 已经增长到约 400 行；它暴露了 12 个 inline tools，并且其中 3 个 tool 实际上是 thin wrappers around hardcoded subagents。换句话说，主 agent 看起来只是“有很多工具”，实际执行时还会绕进多个独立上下文，然后再把 prose 结果返回给 orchestrator。

¹视频来源时间区间：00:02:45–00:04:10。

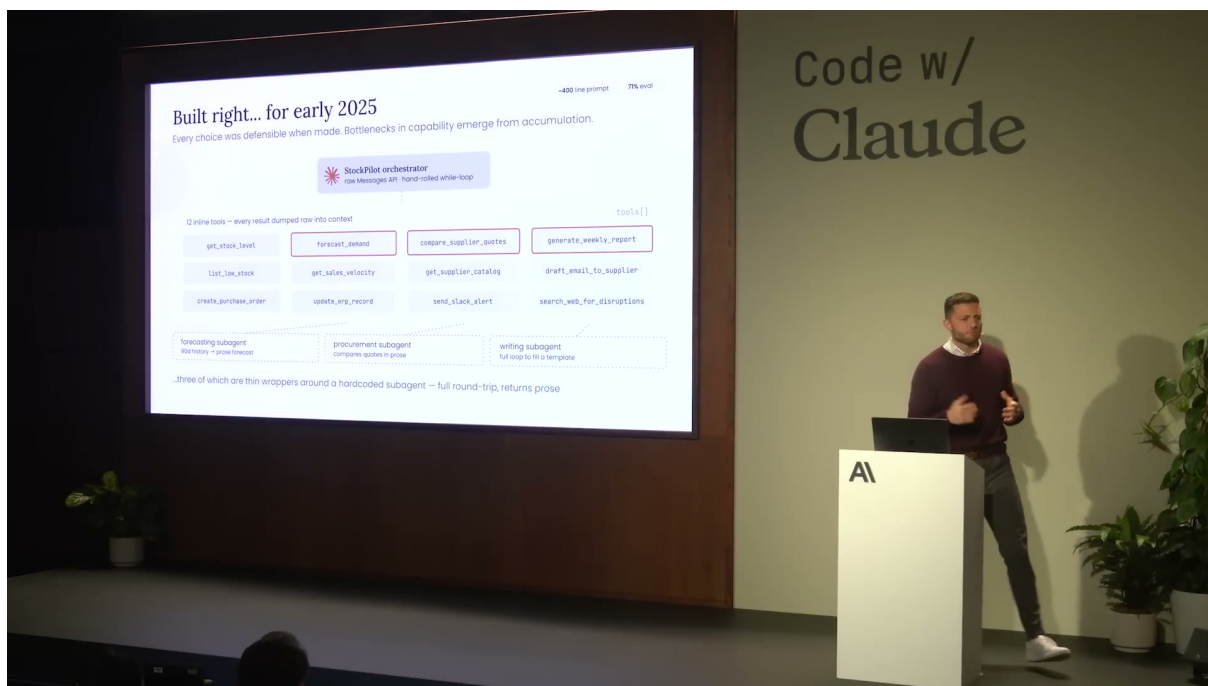


图 2: 早期 Stock Pilot 架构: 一个 orchestrator、约 400 行 prompt、12 个 inline tools, 以及三个包成 tool 的 subagents。²

图中最关键的细节有三层。第一层是顶部的 `StockPilot orchestrator`, 它承担任务接收、tool 调用、subagent 调度和结果整合。第二层是中间的 tool 列表, 例如 `get_stock_level`、`forecast_demand`、`compare_supplier_quotes`、`generate_weekly_report`、`create_purchase_order`、`send_slack_alert` 等。第三层是底部的 forecasting、procurement、writing subagents, 它们以 tool wrapper 形式出现, 完整来回调用后返回 prose。

这个设计在早期 2025 的业务规模下可以成立, 因为每个选择都有局部合理性: 需要预测时加 forecaster, 需要写周报时加 writing subagent, 需要比价时加 procurement subagent。问题在于这些局部合理选择长期叠加后改变了 agent 的认知环境。orchestrator 的上下文变成了规则、政策、流程、工具说明和 subagent 返回格式的混合物; 模型每次决策都要在这堆信息中寻找与当前任务相关的部分。

容易误判的地方

把性能下降直接归因于“模型变弱”会掩盖真正原因。Will 在 R8 例子中明确指出这是 context problem: 模型周围的信息变得过长、互相冲突、难以定位。架构债的症状会表现为 eval dip, 根因常常是信息加载方式和执行边界失衡。

1.4 为什么能力追加会导致 evals 下滑

Stock Pilot 的退化链条可以按第一性原理拆成四步。

1. 业务需求增加: 从低库存检测扩展到 forecasting、supplier selection、purchase order、weekly report。

²视频来源时间区间: 00:04:10–00:05:00。

2. 实现方式追加：把新规则写进 system prompt，把新动作做成 tool，把新复杂任务包成 subagent wrapper。
3. 上下文成本上升：每个任务都携带越来越多不相关政策、工具描述和历史决策边界。
4. 评测表现下滑：原本加速的领域出现 regression，特别是效率、通信和政策一致性相关任务。

这一链条解释了全篇后续的拆解顺序。Section 2 会先引入 evals，把“感觉 agent 变差了”转化为 F1、F2、R8 这样的可定位证据；后续章节再分别处理 prompt 过长、tool surface 过宽、subagent 通信不透明等问题。这个顺序很重要，因为没有 eval baseline，架构调整就只能凭直觉；有了 eval failure，修改才能变成可验证假设。

从问题设定到架构拆解

Stock Pilot 的拆解目标是把责任放回合适边界：长期业务知识进入 skills，通用行动能力收敛为 primitive tools，真正需要隔离上下文的 forecasting 才保留为 subagent 或 callable agent。这个判断必须由 eval evidence 驱动。

1.5 本章小结

本章建立了整个 PDF 的问题起点：Stock Pilot 原本能处理库存管理中的多项任务，后来因为需求持续追加，形成了 single orchestrator、约 400 行 system prompt、12 个 tools 和 3 个 wrapped subagents 的遗留架构。这个架构的风险主要来自知识、行动和独立认知被混在同一执行面里。下一章需要用 evals 把这种架构债转化为可观察证据，再决定 tool、skill 和 subagent 各自应该承担什么。

2 用 evals 把退化变成可定位证据

2.1 结论先行：架构修改必须先绑定 eval evidence

Stock Pilot 的拆解不能从“喜欢哪种架构”开始，而要从 evals 开始。evals 把 agent 的退化从模糊感觉变成可复现样本：F1 暴露低库存扫描的低效路径，F2 暴露 subagent 与 orchestrator 之间的通信断裂，R8 暴露长 system prompt 中政策冲突引发的 promo-month forecast 错误。后续关于 skill、tool、subagent 的选择，都应该服务于这些可观察失败。

本章主结论

evals 是 agent decomposition 的证据主线。没有 baseline，架构调整只是主观重构；有了失败样本、grader 和复跑结果，调整才成为可检验的工程假设。Will 所说的 hill climbing，就是沿着 eval pass rate、token、cost、latency 和质量信号逐步爬升。

本节覆盖 00:05:00–00:11:30 的内容。Will 在这段中先说明 eval suite 的结构，再挑出三个失败 lens，最后引出 hill climbing：先跑 evals 得到 baseline，再修改 agent 架构，复跑 evals，比对指标，然后继续迭代。

2.2 12 个 eval tasks 与 5 类 grader

视频中的 eval suite 包含 12 个任务，覆盖 regression tasks 和 failure mode tasks。R 开头的任务更接近单轮业务回归，例如 stock lookup、below reorder point、create PO、reorder recommendation、promo-month forecast、weekly report。F 开头的任务更像故障模式压力测试，例如 daily low-stock sweep、promo reorder with forecast、batch low-stock alerts。

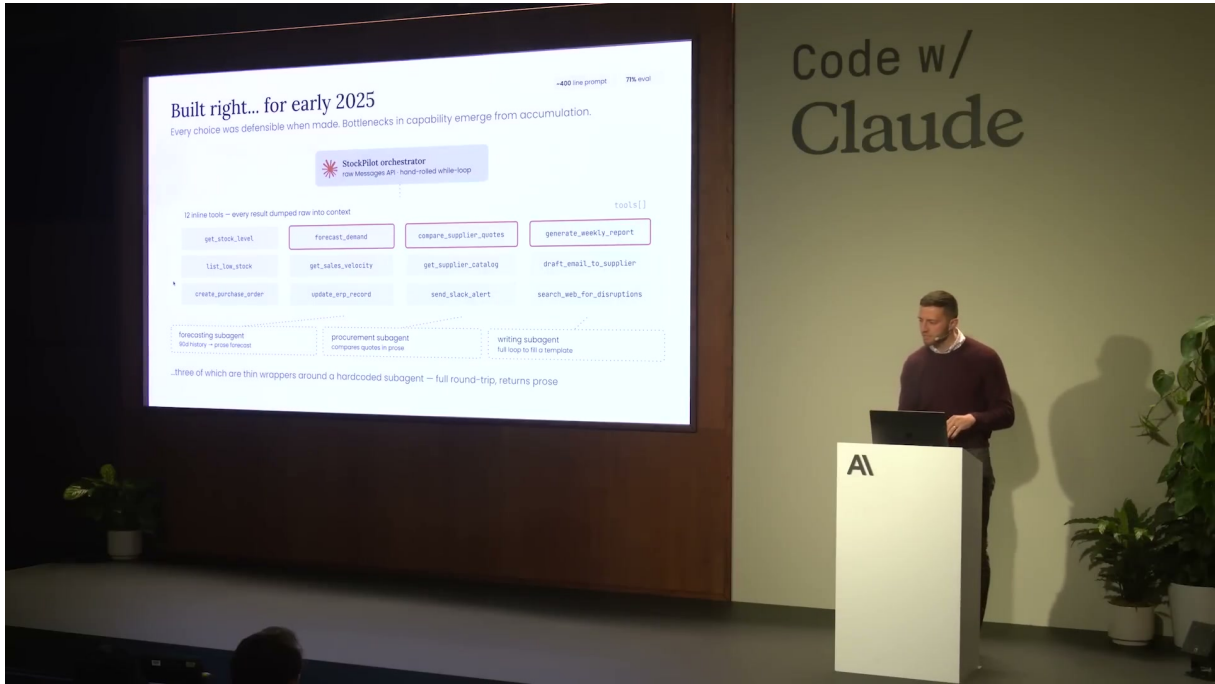


图 3: Stock Pilot 的 eval suite: 12 个任务覆盖 regression 与 failure mode, 并使用 exact match、set match、numeric tolerance、efficiency、LLM judge 等 grader。³

这张表给出两个关键信息。第一，任务同时测答案、路径、效率、输出结构和非确定性质量。第二，grader 类型按任务目标选择：库存查询可以用 `exact_match`，低库存 SKU 列表可以用 `set_match`，预测值可以用 `numeric_tolerance`，例行告警可以用 `efficiency`，周报这种文本质量则需要 `llm_judge`。

表底部给出一个允许部分通过的评分公式。它的作用在于说明 eval suite 如何把慢通过和完全通过区分开。

$$\text{Score} = \frac{\text{PASS} + 0.5 \times \text{PASS-SLOW}}{12}$$

- PASS: 任务正确完成，并满足该 grader 的主要约束。
- PASS-SLOW: 任务完成但效率不足，因此只计半分。
- 12: 当前 eval suite 的任务总数。

这个公式给了 F1 失败一个合理位置：agent 可能最终找到正确低库存项，却因为路径过长、wall time 超限或 token 使用过高而被记为失败或慢通过。对业务系统而言，正确但极慢的 agent 仍然会产生运营成本。

³视频来源时间区间：00:05:00–00:08:50。

deterministic 与 non-deterministic grader

deterministic grader 评估可直接计算的指标，例如 turn count、latency、token usage、路径是否存在、数值是否落入容忍区间。non-deterministic grader 使用 LLM-as-judge 评估 tone、style、report quality 等难以用单个规则表达的质量。两者互补：前者稳定、可重复，后者能覆盖真实文本输出中的语义质量。

2.3 三个 failure lens：F1、F2、R8

Will 挑出的三个失败样本分别对应三类架构问题。

ID	任务	表面失败	架构信号
F1	Daily low-stock sweep	能到达正确结果，但路径绕远，效率不达标	tool 设计没有给模型足够直接的数据处理能力，导致它用高成本方式完成任务
F2	Promo reorder with forecast	subagent 得到正确局部结果，orchestrator 汇总失败	subagent 与 orchestrator 的通信协议、结构化输出和可观测性不足
R8	Promo-month forecast	使用错误 promo multiplier，预测值偏离目标	长 prompt 中分散政策互相冲突，模型在上下文中锚定到错误规则

这三个 lens 把后续三类修复方向提前铺好。F1 指向 tool surface 和 primitive tools；F2 指向 subagent 边界、通信和 observability；R8 指向 system prompt 与 skill 的分工。它们是同一个架构债在不同任务上的投影。

不要把 eval failure 只当成错误清单

eval failure 的教学价值在于定位机制。F1 说明行动能力路径过重；F2 说明跨上下文通信失真；R8 说明 prompt policy 冲突会改变模型取数和计算的锚点。

2.4 R8 细读：promo multiplier 错误怎样暴露 context problem

R8 检查 promotion month 下的 forecast。字幕给出的关键细节是：agent 已经拉到了正确的 forecasting baseline，即每天 12 units；也拉到了正确 promotion multiplier，即 $3.1\times$ 。错误发生在计算阶段，agent 没有使用 $3.1\times$ ，却用了 1.35。如果用 30 天窗口做直观估算， $12 \times 30 \times 1.35 = 486$ units；使用 $3.1\times$ 会产生显著更高的目标量级。outline 中记录的目标约 900 units 与 486 units 的差异，应理解为 failure evidence，具体容忍区间由 eval fixture 控制。

R8: promo-month forecast for SKU-0116

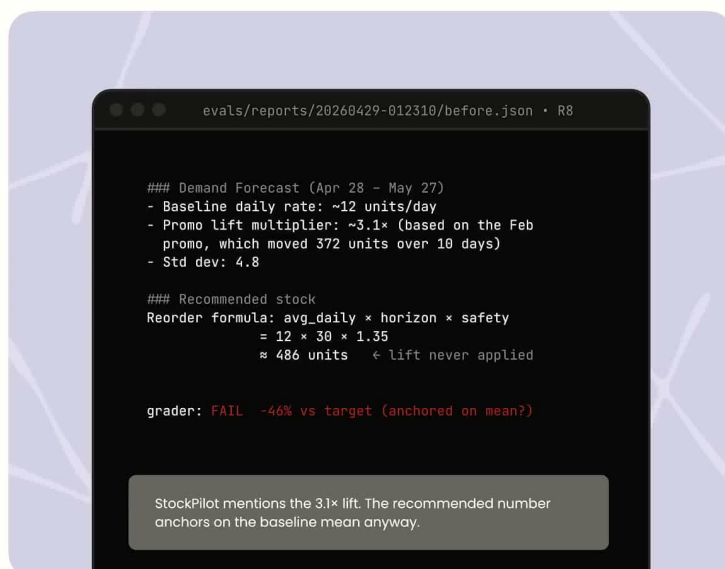
Baseline ~12 units/day. History shows a 3.2× promo spike 60 days ago.

Target = $12 \times 30 \times 2.5 = 900$ units.

What happened?

The reorder formula, the promo policy, and the forecast interpretation live in different sections of a ~400-line prompt.

The agent reads the subagent's prose, extracts "~12/day," and applies the formula. The lift is mentioned but never makes it into the arithmetic.



```
evals/reports/20260429-012310/before.json • R8

### Demand Forecast (Apr 28 - May 27)
- Baseline daily rate: ~12 units/day
- Promo lift multiplier: ~3.1× (based on the Feb
  promo, which moved 372 units over 10 days)
- Std dev: 4.8

### Recommended stock
Reorder formula: avg_daily × horizon × safety
                 = 12 × 30 × 1.35
                 ≈ 486 units ← lift never applied

grader: FAIL -46% vs target (anchored on mean?)

StockPilot mentions the 3.1× lift. The recommended number
anchors on the baseline mean anyway.
```

图 4: R8 的具体失败页：正确证据里包含约 3.1× promo lift，但推荐库存计算仍使用 1.35，导致结果约为 486 units 并触发 grader failure。⁴

R8 的主要矛盾是上下文治理。长 system prompt 中存在位于不同位置的 policy，模型在检索和组合规则时被错误 multiplier 锚定。这个例子说明长 prompt 的危险包含 token 成本、规则优先级、适用范围和局部例外维护困难。模型表面上“看过所有规则”，执行时却可能取用冲突规则。

R8 的工程含义

当政策冲突导致预测失败时，修复方向应该是降低上下文混杂度：把促销预测规则、库存政策和报告模板拆成按需加载的 skills，并让 orchestrator 在任务需要时调用相关知识。这样可以减少每个任务都携带全部业务规则造成的 context pollution。

2.5 Hill climbing: 从 baseline 到可验证改进

Will 把后续工作称为 hill climbing towards eval improvement。这个说法很克制：它表示在当前 eval coverage 下做局部迭代。每一轮先得到 baseline，再提出架构修改假设，复跑 evals，比对 pass rate、token、cost、latency、turn count 和质量指标，然后保留带来净收益的变化。

⁴视频来源时间区间：00:09:30–00:10:20。

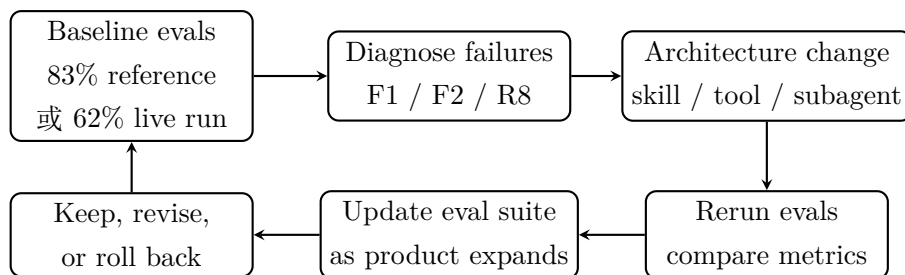


图 5: Hill climbing 的局部迭代结构：baseline、诊断、修改、复跑、比较，再把 eval suite 随产品目标更新。

这张图有两个限制需要保留。第一，hill climbing 依赖 eval coverage；如果 evals 没有覆盖真实产品目标，分数上升也可能掩盖未测风险。第二，baseline 数字必须带上下文。视频中提到 README 或 reference 场景下 up front 约 83%，现场 Claude Code 运行时出现 62%，最终架构后约 92%。这三个数字不能混成同一个指标，它们分别属于不同阶段和运行上下文。

Hill climbing 的边界

eval score 上升只能证明“在当前 eval suite 上变好”。当产品能力扩展时，evals 也必须更新，否则 agent 可能在旧任务上爬升，却在新业务场景中退化。架构优化和 eval 设计是一组联动工作。

2.6 本章小结

本章把 Stock Pilot 的架构债转化为可定位证据：12 个 eval tasks 和 5 类 grader 共同衡量正确性、效率、数值容忍、结构和文本质量；F1 指向 tool 路径低效，F2 指向 subagent-orchestrator 通信断裂，R8 指向长 prompt 中政策冲突。Hill climbing 给出后续章节的工作方法：先建 baseline，再按失败机制调整 skill、tool 或 subagent，复跑 evals，并随着产品愿景扩展评测范围。

3 迁移到 Claude Managed Agents：先把基础设施问题卸载掉

本章的结论很直接：在讨论 tool、skill、subagent 之前，Stock Pilot 先要把自建 agent harness 迁移到 **Claude Managed Agents** (简称 CMA)。CMA 在这里承担的是基础设施边界：它把 agent definition、session details 和 sandboxed tool environment 分层管理，让 workshop 的主要变量回到 agent 架构本身。

本章核心判断

CMA 解决的首要问题是运行边界和基础设施复杂度。它让作者少处理 hosting、scaling、security、memory、observability 这些横向问题，把工程判断集中到三个纵向问题：哪些知识进入 skills，哪些行动能力进入 tools，哪些任务需要独立的 subagent。



图 6: Will 用 CMA 概念页说明: Claude Managed Agents 提供 harness 与 managed infrastructure, 让团队把注意力放回 agent 架构选择。⁵

3.1 从 Messages API 手写循环到 CMA starter

视频中的初始 agent 是基于 Messages API 自己搭出的 agent loop。这个形态适合本地实验: 开发者可以快速写一个循环, 拼接 system prompt, 暴露若干工具, 再把模型响应接回业务逻辑。问题出现在 agent 要被多人、远程、持续地使用时: 同一个循环需要同时处理会话生命周期、工具执行隔离、用户并发、安全边界、日志、状态恢复和成本观测。

这就是 Will 引入 CMA 的动机。CMA 不能自动完成架构设计; 它提供一个托管运行层, 让团队避免在 workshop 里同时优化两类问题:

- **基础设施问题:** agent 如何托管、如何扩展到很多用户、如何隔离工具执行、如何记录 session 和指标。
- **agent 设计问题:** 知识、行动能力、独立认知应该分别放在 system prompt、skills、tools、sub-agents 或 callable agents 的哪一层。

类比: CMA 像把实验台换成托管实验室

手写 Messages API loop 像在个人桌面上搭实验台, 灵活且容易启动。CMA 像把实验搬进有隔离间、审计记录和标准供电的托管实验室。实验内容仍然由研究者设计, 实验室负责把安全、并发、记录和运行环境变成稳定边界。

3.2 CMA 的三层分离

Will 对 CMA 的解释可以压缩成三层: agent definition、session details、sandboxed environment。三层分离后, 架构讨论变得更清楚: agent definition 描述角色、规则和可用能力; session details 记

⁵ 视频画面时间区间: 00:11:31–00:13:10。

录一次用户交互的上下文和运行状态；sandboxed environment 承载 tool calls、文件系统、代码执行等外部动作。

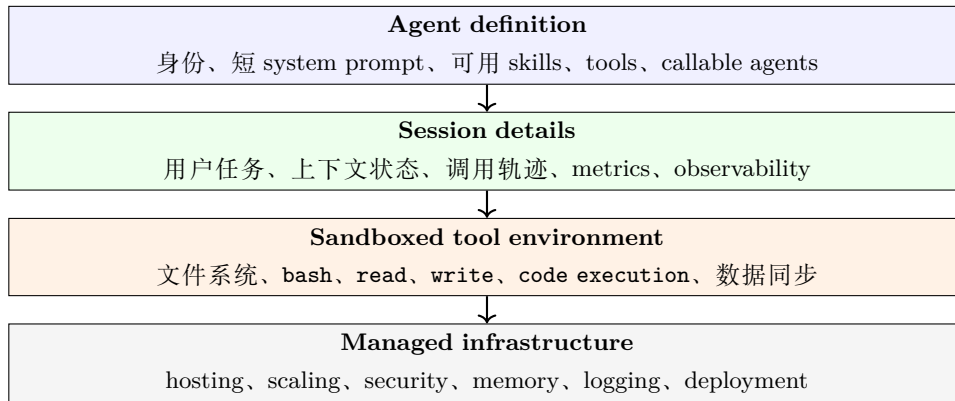


图 7: CMA 在 workshop 中承担的边界：把 agent 设计、会话、工具执行环境和托管基础设施拆成可分别讨论的层。

这个图的教学价值在于避免一个常见误解：CMA、evals 和 agent decomposition 属于不同层次。CMA 让基础设施变量先退出主战场。后续章节讨论 skills、primitive tools 和 subagents 时，读者需要把这些选择看成 agent definition 内部的设计选择，避免把它们理解成 hosting 层面的补丁。

容易误解的地方

把 agent 搬到 CMA 并不自动修复 400 行 system prompt、工具过多、subagent 通信断裂或 R8 policy 冲突。CMA 只是提供更清晰的运行边界。真正的性能修复仍然依赖 evals、诊断、修改架构、重新运行 evals 这一条 hill climbing 链路。

3.3 Workshop 的实验设置

这一段视频把 hands-on lab 的准备步骤讲清楚。学习者打开 workshop URL 后进入 agent decomposition 文件夹，先 clone repo，然后运行 `uv sync` 安装依赖。接着，学习者需要在 Claude Console 创建 API key，把它写入由 env example 复制出的环境文件。完成这些准备后，实验会围绕两套 agent 展开：

- **before**: 基于 Messages API 的自建 agent loop，用于呈现旧架构的真实问题。
- **starter**: 已经部署到 CMA 的起始版本，用于后续使用 Claude Code 比较、修改和重新部署。

Time to get building!

To get started, work through the following tasks and measure improvements using the eval suite.

After every edit: `uv run deploy starter` then re-run evals. Your agent config lives on CMA.

Ask questions or pair up if you get stuck anywhere!

Repo: <https://cwc26.short.gy/workshops>

- 1 Swap to SHORT_PROMPT + enable notify / weekly skills.

```
uv run evals --task F3
```
- 2 Drop the data-dump tools, setup CMA with bash

```
uv run evals --task F1
```
- 3 Implement subagent(s) for forecasting and token efficiency.

```
uv run evals --task F2,R7
```

图 8: Hands-on workshop 的 setup 步骤: clone repo、运行 `uv sync`、配置 API key、跑 baseline evals 并部署 starter agent。⁶

本节最重要的命令有两条。第一条用于在旧 agent 上跑 baseline:

```
1 uv run evals --agent before
```

Listing 1: 在旧 Messages API agent 上运行 evals 的命令

第二条用于部署 CMA starter:

```
1 uv run deploy starter
```

Listing 2: 部署 CMA starter agent 的命令

这些命令的意义在于展示 workshop 的变量控制: 先拿旧架构跑 baseline, 再把可运行的 CMA starter 作为改造对象, 之后每一次架构变化都通过 evals 回到可观测指标。工具链细节服务于这个实验结构。

为什么要保留 before 和 starter 两套 agent

before 保留了 400 行 prompt、12 个 tools 和 3 个 wrapper-style subagents, 适合做故障标本。starter 把运行层迁到 CMA, 适合做改造实验台。两者并存让读者能比较“旧问题如何出现”和“新架构如何逐步吸收修复”。

3.4 CMA 如何为后续拆解创造边界

迁移完成后, workshop 的讨论顺序变得有章法。Will 先让 Claude Code 跑 evals 并诊断失败, 再逐一处理三类架构债:

1. system prompt 太长, policy 和流程知识常驻 **context window**, 导致信息污染和 policy 冲突。

⁶视频画面时间区间: 00:13:50–00:16:30。

2. tools 太多且形态僵硬，很多数据分析工作被迫塞进模型上下文，造成 token、成本和 wall time 膨胀。
3. subagents 被包装成普通 tools，通信结果、日志和 observability 容易丢失。

这三类债务分别对应后续三章：Section 4 用 skills 和 progressive disclosure 拆 prompt；Section 5 用 primitive tools、custom tools 和 MCP 决策梯重整行动能力；Section 6 再决定哪些 subagent 值得保留为 CMA callable agents。

承上启下

CMA 的作用是把“agent 怎么稳定运行”这件事先标准化；skills、tools、subagents 的作用是把“agent 怎么聪明地工作”这件事拆清楚。后续章节的所有改造都应该回连到 eval evidence，避免只凭架构偏好做选择。

3.5 本章小结

本章说明了为什么 workshop 要先迁移到 Claude Managed Agents。Messages API 手写 loop 适合本地原型，CMA 适合把 session、sandbox、deployment、observability 和安全边界托管起来。完成这一步后，Stock Pilot 的改造焦点从基础设施负担转向 agent decomposition：按需知识进入 skills，行动能力进入 tools，需要隔离认知的任务进入 subagent 或 CMA callable agents。这个边界为下一章的 skills 改造奠定了前提。

4 第一刀：把长期 system prompt 拆成 skills

本章的结论是：当 system prompt 承载了大量只在部分任务中才需要的政策、流程和模板时，正确的第一刀是把这些知识迁入 **skills**，用 progressive disclosure 让模型按需读取。Stock Pilot 的失败证据显示，长 prompt 带来了 context pollution、policy 冲突和输出结构混乱；skills 把这些信息从“每轮常驻”改成“任务触发时加载”。

本章核心判断

skill 解决的是“按需知识”问题。system prompt 应保留每个任务都需要的身份、边界和不可变约束；skills 承载只在某些任务中才需要的流程、政策、模板、测试规则和业务知识。

4.1 失败诊断：62% 分数暴露四类问题

Will 在 Claude Code 中先运行 `uv run evals --agent before`，再让 Claude 总结失败原因。画面显示 live run 的结果是 7/12 正确，也就是 62%。这比前文 README 预期的 83% 更差，原因集中在四类问题：F1 的低库存扫描耗时且 token 过高，R2 的 reorder point 查询漏 SKU，F2 的 promo reorder 输出结构不符合要求，R8 的 promo-month forecast 忽略了正确的 promo lift。


```
* Worked for 1s

Help me understand the eval results. What are the core issues? Summarize simply.

Read 1 file

• Here's what the before-agent's run tells you, plain:

Score: 7/12 correct, 62% – right at the bottom of the reference range. The failures cluster into four distinct problems, each pointing at a different fix:

1. F1 – "Daily low-stock sweep" (fail, 328s / 216k tokens)
Wall-time blown. The agent is probably calling list_low_stock or similar per-SKU tools and dragging 400 rows of product data through context before deciding anything. Classic compute-over-context miss – the filtering should happen in a tool, not in the model.

2. R2 – "Below reorder point" (fail, missed 2 SKUs)
Related to F1: the low-stock query isn't reliably returning the full set. Same root cause – raw CSV-sized data in context, model sampling rather than computing.

3. F2 – "Promo reorder w/ forecast" (fail, qualitative confidence)
The output isn't in the required shape – confidence came back as prose instead of a number. That's a typed-contract failure. A forecasting skill with a JSON schema in its SKILL.md would fix it.

4. R8 – "Promo-month forecast" (fail, -34% vs target)
Forecast anchored on the mean and ignored the promo lift. Same skill (forecasting) would carry the policy for how to handle promos.

5. F3 – "Batch low-stock alerts" (pass-slow, 15k out-tokens vs 5k budget)
Correct answer, but the model is writing the alert text itself instead of using a fixed template. That's policy-as-skill – notify-templates gives it the exact format and stops it from re-generating the wording every time.
Jump to bottom (ctrl+End) ↓

Understand eval results and core issues

▶ auto mode on (shift+tab to cycle) • ← for agents
```

图 9: Claude Code 对 before-agent eval 结果的诊断: 7/12 正确、62%, 失败被聚类到 compute-over-context、结构化输出、policy 冲突等问题。⁷

这张图把 Section 4 的修复对象限定得很清楚: 修复目标是两类由 prompt 承担过多责任引发的问题。第一, 模型被迫在上下文里阅读和推理大量业务规则; 第二, 不同时间追加的 policy 可能分散在 prompt 的不同位置, 造成 R8 这类 promo multiplier 冲突。

不要把失败都归因给模型能力

R8 的关键线索是 context problem。agent 已经能取到正确 baseline 和正确 promo multiplier, 失败来自系统周围的信息组织方式。若把它误判成模型能力不足, 后续会倾向于换模型或加更多 prompt, 反而继续扩大 context window 压力。

4.2 skill 的定义: packaged and composable information

Will 给出的短定义是: skills 是 packaged and composable information。这个定义的重点是“信息的封装方式”和“加载时机”。tool 提供行动能力, subagent 提供隔离上下文和独立认知, skill 的责任是一组能被模型按任务需要拉入上下文的说明材料, 例如:

- reorder policy: 补货规则、promo handling、escalation matrix。
- forecasting guideline: 预测流程、返回 JSON schema、置信度表达方式。
- report template: 周报结构、固定措辞、输出字段。
- supplier selection rules: 供应商排序、报价比较、异常处理。

⁷ 视频画面时间区间: 00:18:20–00:20:30。

相关概念：system prompt、skill、context window

system prompt 像员工每天进办公室都要背下来的岗位纪律；skill 像放在知识库里的 SOP，任务需要时再打开；context window 是员工当下能摆在桌面上的所有材料。把所有 SOP 都塞进 system prompt，会让桌面一直拥挤，且新旧规则容易互相覆盖。

4.3 progressive disclosure：只在需要时支付上下文成本

图中 Claude Code 的分析把 progressive disclosure 机制讲得很具体：CMA 只把每个 skill 的 description 字段放进 standing context，完整 SKILL.md body 只有在模型判断任务匹配时才读取。也就是说，普通 stock lookup 不需要为 reorder policy 付出完整上下文成本；真正的 reorder task 才加载相关 skill。

```
what can we do about the tools? leaning into bash, read, and write instead
└─ /plan to preview

Let's start with #1. Look at @agents/starter/agent.py. Any thoughts on the system prompt? Can I use Skills instead of a long system prompt for progressive disclosure?
└─ Read agents/starter/agent.py (126 lines)

• Yes – this is cycle 1. Same picture as before:

Why the long prompt is wasteful
- LEGACY_PROMPT is 416 lines, loaded on every turn – even for a 17k-token stock lookup (R1) that never touches reorder rules, supplier ranking, or report structure.
- The five .claude/skills/ files already cover that same policy territory. reorder-policy/SKILL.md alone has ~35 sections (decision rules, promo handling, escalation matrix...) – all currently duplicated inside LEGACY_PROMPT.

How skills give you progressive disclosure
- CMA loads only each skill's description: field (~one line, ~50 tokens) into standing context.
- The full SKILL.md body is read only when the model decides the task matches – so a stock lookup pays nothing for reorder policy, but a reorder task pays the full cost exactly when it's useful.
- That one-line description: is also what triggers the skill – it's the model's index of "when would I need this."

Why the prompt swap is paired with enabling skills
SHORT_PROMPT (already drafted at agent.py:66-84) explicitly says "business policies live in skills – load the relevant one before applying a policy." Swapping prompts without enabling skills would leave the model with no policy at all. Enabling skills without swapping the prompt would duplicate every policy (once in the 416-line preamble, once in the skill).

Framework principle: policy-as-skill + progressive disclosure.

Making the edit – and this time also dropping the now-unused LEGACY_PROMPT import (Pyright flagged it last round):

• Update(agent.py)
└─ Removed 1 line
1 new message (ctrl+End) ↓

Deploy and run the suite
Understand eval results and core issues

auto mode on (shift+tab to cycle) · ← for agents
```

图 10: Claude Code 对 progressive disclosure 的解释：LEGACY_PROMPT 常驻每轮，skills 只让短 description 常驻，完整 SKILL.md 按任务触发加载。⁸

下面的上下文预算图把这个机制再压缩一次。横轴表示相对信息量，用来呈现“每轮默认加载的信息量”和“任务触发后才加载的信息量”的责任差异。

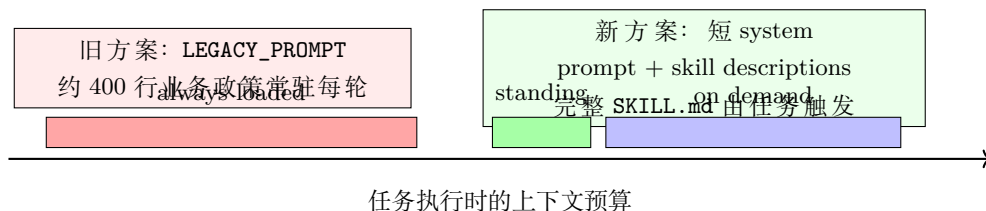


图 11: context-budget 示意：skills 把长期常驻信息拆成短索引和按需正文，减少无关任务的上下文污染。

⁸视频画面时间区间：00:21:17–00:25:15。

4.4 从长 prompt 到短 prompt：边界如何划分

Stock Pilot 的 prompt swap 目标高于长 prompt 删除。Claude Code 的分析指出，短 prompt 需要显式声明 “business policies live in skills, 应用 policy 前先加载相关 skill”。如果只缩短 prompt 却不启用 skills，模型会缺少业务规则；如果只启用 skills 而保留长 prompt，同一政策会在 prompt 和 skill 中重复出现，造成维护和冲突风险。

因此，边界划分可以按三条规则执行：

1. **每个任务都需要的规则留在 system prompt**：身份、权限边界、基本响应原则、必须加载相关 skill 的元规则。
2. **部分任务才需要的知识迁入 skills**：补货、供应商、预测、通知模板、周报结构。
3. **需要执行外部动作的能力留给 tools**：读写文件、执行代码、调用 API、访问数据。

policy-as-skill

当一个规则只在特定任务中被使用时，它更适合作为 skill。长期驻留在 system prompt 中会扩大无关任务的上下文成本。policy-as-skill 的价值在于同时改善可维护性和上下文成本：业务团队可以维护独立 SOP，agent 只在任务命中时读取它。

4.5 对 F2、R8 和模板输出的影响

skills 对失败样本的修复有具体落点。F2 的 promo reorder 需要结构化 confidence，forecasting skill 可以把返回形态固定为 JSON schema，避免 subagent 返回 “fairly confident” 这类自由文本。R8 的 promo-month forecast 需要统一的 promo handling policy，forecasting skill 可以把 multiplier 使用规则集中到一处，减少长 prompt 中分散政策互相冲突的机会。F3 的 batch low-stock alerts 虽然结果正确，却超过输出 token 预算；notify-template skill 能把固定措辞交给模板，避免模型每次重新生成完整告警文本。

skill 的风险

skills 依赖触发描述。description 写得太泛会造成误触发，写得太窄会漏加载。实际项目中需要把 skill activation 也纳入 evals：某个任务失败时，检查模型是否加载了正确 skill，以及是否在正确时机加载。

4.6 本章小结

本章解释了 workshop 的第一刀：把长期 system prompt 中的业务流程、政策和模板迁入 skills。eval triage 给出失败证据，progressive disclosure 给出机制，context-budget 图说明了成本变化。最终边界是：system prompt 保存核心身份和不可变约束，skills 保存按需知识，tools 保存行动能力。这一步直接服务于 R8 policy 冲突、F2 结构化输出和 F3 模板输出成本等失败信号。

5 第二刀:把工具面收敛到 primitive tools、custom tools 和 MCP 决策梯

本章的结论是:tool 的核心含义是 agent 可调用的行动能力。Stock Pilot 的第二刀先回到 human-like **primitive tools**: 文件系统、bash、read、write、code execution、web search、todo list。只有当行动具有稳定领域语义时才加 custom tools; 只有当多个 clients 或 agents 需要共享同一套标准化、受治理的工具集合时才引入 **MCP**。

本章核心判断

工具设计要同时回答三个问题: agent 需要什么行动能力, 行动是否会扩大 context window, 工具集合是否需要跨团队或跨 client 治理。primitive tools 解决通用行动, custom tools 解决本地领域动作, MCP 解决共享和治理边界。

5.1 human-like primitive tools: 把模型放回工作站

Will 的工具原则来自一个直觉: 优秀的 agent 应该先拥有类似人类工作站的基本能力。人类到岗后能浏览文件、搜索网页、写代码、运行脚本、维护待办清单; Claude Code 之所以强, 是因为它把这些基本能力交给了 Claude。随着模型变强, 同一组 primitives 会被使用得更好。

在 Stock Pilot 中, 这个原则尤其适合数据分析任务。旧架构为低库存、供应商、预测、采购单等动作准备了大量专门 tools, 有些 tool 每次返回大块 CSV 或自由文本。模型被迫在 context window 里“用眼睛扫描数据”。新的做法是把数据同步进 CMA sandbox, 再让 agent 用 bash、read、write 和 code execution 直接读取、排序、过滤、写回结果。

类比: 给模型计算器, 避免整本账本进上下文

如果任务是找出库存最低的 20 个 SKU, 把 395 行 CSV 全塞进上下文, 相当于要求一个人背完整本账本再心算排序。给 agent code execution, 相当于给它计算器和表格工具: 它写一个小脚本排序, 只把结果摘要带回推理链路。

5.2 工具反模式: context-dumpers、subagent-hiders、trivial writes

图中 Claude Code 对现有工具做了三类诊断。第一类是 context-dumpers, 例如 list_low_stock 每次把 395 行 CSV 交给模型, F1 timeout 和 R2 漏 SKU 都来自这个模式。第二类是 subagent-hiders, 例如 forecast_demand 和 compare_supplier_quotes 把 subagent 包成 tool, 返回自由文本, orchestrator 再从 prose 中解析数字或 supplier_id。第三类是 trivial writes, 例如 create_purchase_order 只有构造 dict、追加 JSONL、返回 status 这几步, 用 bash 写入 purchase_orders.jsonl 就能完成。

```
Let's start with #1. Look at @agents/starter/agent.py. Any thoughts on the system prompt? Can I use Skills instead of a long sys...
* recap: We're decomposing the StockPilot starter agent; cycle 1 is done (5 skills enabled, SHORT_PROMPT swapped in). Next: run `uv
run deploy starter && uv run evals --agent starter` to confirm F2, R8, F3, R9 flip. (disable recaps in /config)

what can we do about the tools? I want to lean into bash, read, and write instead of rigid tools.

• Cycle 2. Let me ground this before deleting - here's what list_low_stock actually hands back to the model every time:

Searched for 1 pattern, read 1 file, ran 2 shell commands

• Now you can see the three patterns concretely:

1. Context-dumpers - list_low_stock hands 395 rows of CSV to the model every call. The model then has to scan them by eye to find
the top-N urgent SKUs. That's F1 (timeout) and R2 (missed 2 SKUs) in one. A Python one-liner would sort by (on_hand -
reorder_point) and return the top 20 as compact JSON. Compute happens in the sandbox, not in the model's head.

2. Subagent-hiders returning prose - look at forecast_demand: it's literally
return subagents.forecasting_subagent(sku, context_note=note)
compare_supplier_quotes is the same shape. The orchestrator then parses a number or a supplier_id out of freeform text. That's
where F2's confidence gets dropped - the prose says "fairly confident," not 0.72. The forecasting and supplier-selection skills
fix this by pinning a JSON return shape.

3. Trivial writes - create_purchase_order is 3 lines: build a dict, append JSONL, return status. Bash python -c 'json.dumps(...)'
>> sinks/purchase_orders.jsonl does the same thing with no tool-definition overhead.

And one scope call: search_web_for_disruptions is a stub returning canned text. Web/news monitoring doesn't belong in the
inventory agent - that's a separate service.

Framework principles: compute-over-context (for th 1 new message (ctrl+End) ↓ act via skill (for the subagent-hiders - the skill

) deploy and run the evals
▶ auto mode on (shift+tab to cycle) · ← for agents
```

图 12: Claude Code 对 Stock Pilot 工具面的分析: context-dumpers、subagent-hiders 和 trivial writes 指向不同修复路径。⁹

这张图把工具整理的决策依据讲清楚了: 低库存扫描应该 compute-over-context, 在 sandbox 里用代码完成过滤; subagent 返回 prose 的问题应该由 skill 或结构化 schema 约束; 采购单写入这种简单副作用可以由本地文件写入完成, 减少工具定义开销。

工具过多会制造假清晰

专门工具的名字看起来清楚, 实际可能隐藏了大块上下文、自由文本协议和重复业务逻辑。工具越多, 模型越需要在工具列表中做路由, 维护者也越难判断某个失败来自工具实现、参数选择、上下文膨胀还是返回协议不稳定。

5.3 CSV 数据分析: code execution 比整表入上下文更省

视频中的 CSV 例子是 Section 5 的核心机制。若 agent 需要跨大量 CSV 或 Excel 数据做分析, 直接把整份文件塞入 context window 会消耗大量 token, 也让模型承担本该由程序完成的排序、聚合和过滤。更稳的路径是让 Claude 写一个 Python 脚本, 在 sandbox 中运行, 然后只把结果摘要带回上下文。

```
1 import csv
2 import json
3
4 rows = []
5 with open("inventory.csv", newline="") as f:
6     for row in csv.DictReader(f):
7         gap = int(row["reorder_point"]) - int(row["on_hand"])
8         if gap > 0:
9             rows.append({"sku": row["sku"], "gap": gap})
10
```

⁹视频画面时间区间: 00:26:12-00:30:15。


```

11 rows.sort(key=lambda item: item["gap"], reverse=True)
12 print(json.dumps(rows[:20], ensure_ascii=False))

```

Listing 3: 用 code execution 承担低库存排序的示意脚本

这段示意代码体现了 compute-over-context 的含义：计算发生在 sandbox 里，模型只读取排序后的 top-N JSON。它减少 token，也降低“模型抽样看错几行”的风险。

5.4 MCP 的位置：共享、标准化、受治理时再使用

Will 对 MCP 的建议是一套决策梯。CMA 中有三类工具路径：先使用 Claude Code primitives，例如 web search、code execution、file system；再为单个 agent 创建 local custom tools；当多个 agents 或多个 Claude Code clients 都需要访问同一套标准化、受治理的工具集合时，再把它们整理成 MCP server。

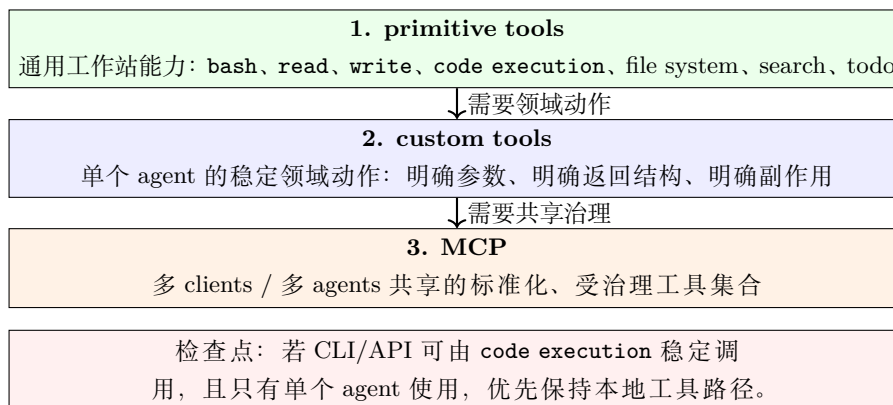


图 13: tool decision ladder: 先 primitives，再 custom tools，最后在共享治理需求明确时引入 MCP。

MCP sprawl 风险

过早引入 MCP 会让工具说明、schema、server 列表占用 context window，并造成多个 MCP server 功能重叠。MCP 的价值来自治理和共享；若只是临时给单个 agent 调一个 CLI 或 API，code execution 往往更轻。

5.5 指标效果：F1 从 FAIL 到 PASS

工具面收敛后，F1 的 before/after 画面给出可量化证据。F1 从 FAIL 变成 PASS，wall time 从 328 秒降到 164 秒，tokens 从 216,365 降到 7,628，turns 从 7 降到 5，cost 从约 2 美元降到约 0.07 美元。这个结果的证据边界很明确：它证明这个任务里 compute-over-context 成立。低库存扫描主要是表格过滤和排序，代码执行比上下文扫描更合适。

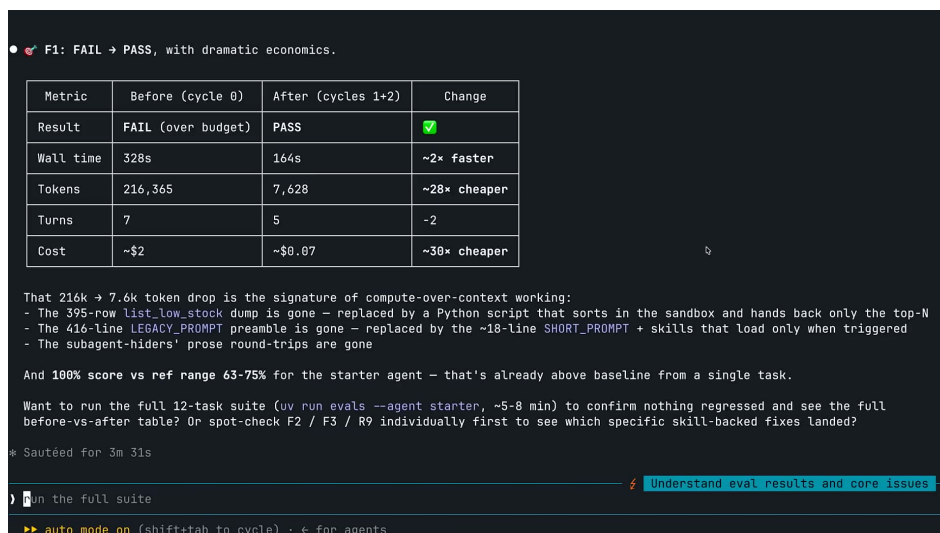


图 14: F1 before/after 指标: 工具收敛和 code execution 让低库存扫描从超预算失败变成通过, 并显著降低 token、时间和成本。¹⁰

指标也提示一个权衡: turn count 和 latency 不一定总是同步下降。对高智能任务, 团队可能接受略高的 turn count, 只要正确率、token、成本和可观测性更好。因此, 工具设计不能只看工具数量, 还要看 eval pass rate、token usage、cost、wall time、latency 和 failure mode 是否一起改善。

把工具选择回连到 evals

每一次工具改造都应该能回答: 它修复了哪个 failure mode, 它减少了哪些 token 或时间成本, 它有没有让新的 failure mode 出现。F1 的 PASS 是强证据; 其他任务仍然需要重新跑完整 eval suite 才能排除回归。

5.6 本章小结

本章说明了 Stock Pilot 的第二刀: 收敛工具面。primitive tools 给 agent 类似人类工作站的基本行动能力, custom tools 只在领域动作稳定且值得封装时加入, MCP 适用于多个 clients 或 agents 共享同一受治理工具集合的场景。F1 的 before/after 指标展示了 compute-over-context 的收益: 用 code execution 处理 CSV, 比把整表塞入 context window 更省 token、更快、更便宜。这个结论仍然受 eval coverage 约束, 完整架构改造需要继续通过全套 evals 验证。

6 第三刀: 只保留有隔离价值的 subagent

本章的结论先给出: subagent 的保留条件是隔离收益高于通信成本。它适合把大问题拆给多个 Claude 实例并行探索, 也适合让一个 fresh mind 独立评审或独立预测; 如果某个能力只是读取数据、执行脚本、补充流程知识, 优先放回 primitive tools 或 skills。在 Stock Pilot 的最终改造里, 作者保留的是 forecasting subagent, 原因是预测过程容易受到主 orchestrator 对话上下文的干扰, 独立上下文能带来更可靠的数字来源和更清楚的 provenance。

¹⁰ 视频画面时间区间: 00:34:00–00:35:30。

本章核心判断

subagent 解决的是“独立认知与上下文隔离”问题；**tool** 解决的是“行动能力”问题；**skill** 解决的是“按需知识”问题。把三者混在一起，会制造额外通信、日志、调试和 eval 归因成本。

6.1 Subagent 的两类强场景

Will 给出的第一类场景是 **parallel exploration**：当任务本身很大、路径很多、需要把大量 Claude 实例同时投向同一个问题时，**subagent** 可以缩短探索时间，并提高覆盖率。深度研究、网页搜索、代码库探索都属于这一类。直觉类比是让多名工程师同时读不同模块、验证不同假设，然后由一个负责人整合结果。

第二类场景是 **fresh mind**：当生成者和评审者需要分离时，另一个没有初始上下文污染的 Claude 实例可以更像代码评审者。写代码的人常常看不见自己的盲点；同理，一个负责生成方案的 agent 也可能被自己的中间假设锁住。新的 **subagent** 不继承原始对话里的所有暗示、纠结和错误路径，更容易发现结构、事实或约束上的问题。

使用场景	典型任务	隔离价值	主要代价
parallel exploration	深度研究、代码库探索、多路径搜索	多个 Claude 实例覆盖不同路径，降低单一路径遗漏风险	结果整合和去重成本上升
fresh mind	代码评审、方案审查、预测验证	评审者不继承生成者的错误前提	需要清晰输入契约和输出结构
forecasting subagent	库存预测、促销月需求估计、置信度输出	预测上下文与客户对话上下文分离	主 agent 与子 agent 通信必须可观测

这两类场景共享同一个底层条件：独立上下文能产生实质收益。若只是为了“看起来模块化”而拆出 **subagent**，系统会在每一次调用时支付通信、日志和调试成本。

6.2 Wrapped subagent 的风险

Stock Pilot 原本有 12 个工具，其中 3 个本质上是 **subagent wrapper**：主 **orchestrator** 看到的是一个工具调用，工具背后再启动另一个 Claude 实例。这个形态看起来统一，因为所有能力都像工具一样暴露；问题在于它把关键过程藏在工具边界之后。

Tool wrapper 会遮住三类证据

第一，**通信证据**会变弱：主 **orchestrator** 发给 **subagent** 的任务是否完整，回传内容是否丢失，往往很难从主 session 直接判断。第二，**日志证据**会分散：多个 Claude 实例各自产生 transcript，调试者需要额外收集和对齐。第三，**eval 归因**会变难：当 F2 或 R8 失败时，失败可能来自主 agent 的提示、子 agent 的推理、wrapper 的参数映射，或三者之间的翻译损耗。

Will 用人类协作作类比：一个人说出想法，另一个人可能按自己的方式理解。agent 系统也一样，**orchestrator** 给出的任务描述、**subagent** 接收到的上下文、以及返回结果的结构都可能出现偏差。随着子代理数量增加，偏差还会互相叠加。

6.3 为什么保留 forecasting subagent

经过前两刀改造后，许多原先被包装成工具或 subagent 的能力已经回到 `bash`、`read`、`write` 和 `skills`。库存预测被保留下来，因为它需要“隔离的判断环境”。客户和主 `orchestrator` 的对话里可能包含临时请求、解释性文字、策略讨论和不完整假设；如果同一个 Claude 实例直接在这堆上下文里做预测，预测值可能被非预测信息牵引。

```
110 ],
111 +   "wants_forecaster": True, # provision + attach the forecaster via callable_agents
112 }
```

● Diff narration: one line. `deploy` will now provision `stockpilot-forecaster` (`bash+read` only, its own context window, same mounted CSVs) and attach it to the starter via `callable_agents`. The forecasting skill's Path B becomes live.

Expected eval movement:

- F2 (promo reorder w/ forecast, numeric confidence) – should PASS with cleaner provenance: the number now comes from a subagent that returned `{"forecast_qty": 1240, "confidence": 0.72, "method": "rolling_mean_with_promo_lift", "flags": ["promo_next_month"]}` rather than being extracted from prose
- R8 (promo-month forecast) – same mechanism; should stay PASS but via Path B delegation rather than inline computation
- Main agent's token cost on forecast-triggering tasks should drop a bit – it no longer loads 90d of history into its own context

Want to deploy + run F2 to confirm the `callable_agents` path lights up, or first open `.claude/skills/forecasting/SKILL.md` and walk through the A/B routing rules that decide when the main agent delegates vs. computes inline?

* Baked for 22s

! Nice! Let's fix the subagent

● Deploying with the forecaster attached, then running F2 – the cleanest test of the typed-contract path.

● Running 1 shell command...

```
L $ uv run deploy starter (20s)
  (ctrl+b to run in background)
```

+ Galloping... (24s · ↓ 364 tokens · running tool for 20s)

```
L Tip: Did you know you can drag and drop image files into your terminal?
```

! Understand eval results and core issues

▶ auto mode on (shift+tab to cycle) · ← for agents · esc to interrupt

图 15: 在 CMA 中通过 `wants_forecaster` 挂载 forecaster，并用 `callable_agents` 路径让预测结果携带结构化 provenance。¹¹

图中最关键的变化是：`wants_forecaster: True` 触发平台 provision 一个 `stockpilot-forecaster`，并通过 `callable_agents` attach 到 starter。这个 forecaster 只拿必要工具，例如 `bash` 和 `read`，拥有自己的 context window，同时读取同一批 mounted CSVs。主 `orchestrator` 负责判断任务是否需要预测；真正的预测计算由独立 forecaster 产生结构化结果。

Skill 与 forecasting subagent 的配合

skill 负责把预测流程、业务规则和 A/B routing guideline 按需加载进上下文；forecasting subagent 负责在隔离上下文里执行预测。两者是互补关系：前者给步骤和标准，后者给独立执行环境。

这条路径直接对应 eval 证据。图中写明，F2 这类“带预测的促销补货”任务应当因为更干净的 provenance 而通过：数字来自 subagent 返回的结构化字段，例如 `forecast_qty`、`confidence`、`method` 和 `flags`，自由文本抽取不再承担数字来源。R8 这类促销月预测也走同一机制，通过 Path B delegation 保持通过。主 agent 在触发预测任务时还会少加载一部分历史数据，token 成本有下降空间。

¹¹ 视频画面时间区间：00:38:20–00:39:30。

6.4 Callable agents 带来的通信、日志与 observability

callable agents 是 CMA 提供的 native managed subagents 能力。它和 tool-wrapped sub-agent 的差异在于观测边界：平台能够把子代理的 session-level observability、metrics 和 transcript 作为同一套托管会话的一部分来管理，主 orchestrator 不必把子代理伪装成普通工具再自行拼接日志。

Callable agents 的工程价值

callable agents 把“需要独立上下文的 Claude 实例”变成平台原生能力。它保留 subagent 的隔离价值，同时改善通信记录、日志归集和 eval 追踪。对 Stock Pilot 来说，这让 forecasting 保持独立，又让 F2/R8 这类 eval 的失败或通过更容易归因。

这里的设计准则可以压缩成三问。第一，任务是否需要独立推理或独立评审？第二，主 agent 与子 agent 的输入输出是否能形成明确契约？第三，日志和 metrics 是否能让调试者看到子代理实际做了什么？三问都成立时，subagent 才有保留价值。

6.5 本章小结

第三刀没有追求“更多 agent”，它追求的是更清楚的认知边界。并行探索和 fresh mind 评审是 subagent 的强场景；普通数据处理、脚本执行和流程知识更适合回到 primitive tools 与 skills。Stock Pilot 最终保留 forecasting subagent，因为预测任务需要隔离上下文，同时通过 CMA 的 callable agents 获得更好的 communication、logging 和 observability。

7 总结与延伸：一套 agent decomposition 决策框架

全场 workshop 的最终结论可以压缩成一句话：agent decomposition 的目标是让知识、行动能力、独立认知和 eval 证据各自回到可管理边界。Stock Pilot 从一个 400 行左右 system prompt、12 个工具、3 个 hardcoded subagents 的膨胀架构，收束到一个部署在 CMA 上的 orchestrator、约 15 行 system prompt、按需加载的 skills、bash/read/write 工具集、以及通过 callable_agents 挂载的 forecaster。最终 eval score 约为 92%，同时 token、cost 和 execution time 明显改善。

最终架构的一句话

一个主 orchestrator 负责接收任务、调用工具、加载 skills 和整合结果；业务流程进入按需加载的 skills；通用行动能力交给 bash/read/write；需要隔离上下文的预测交给 CMA callable agents；所有改动都回到 evals 验证。

One agent. Five skills. Subagents only when a skill says so.

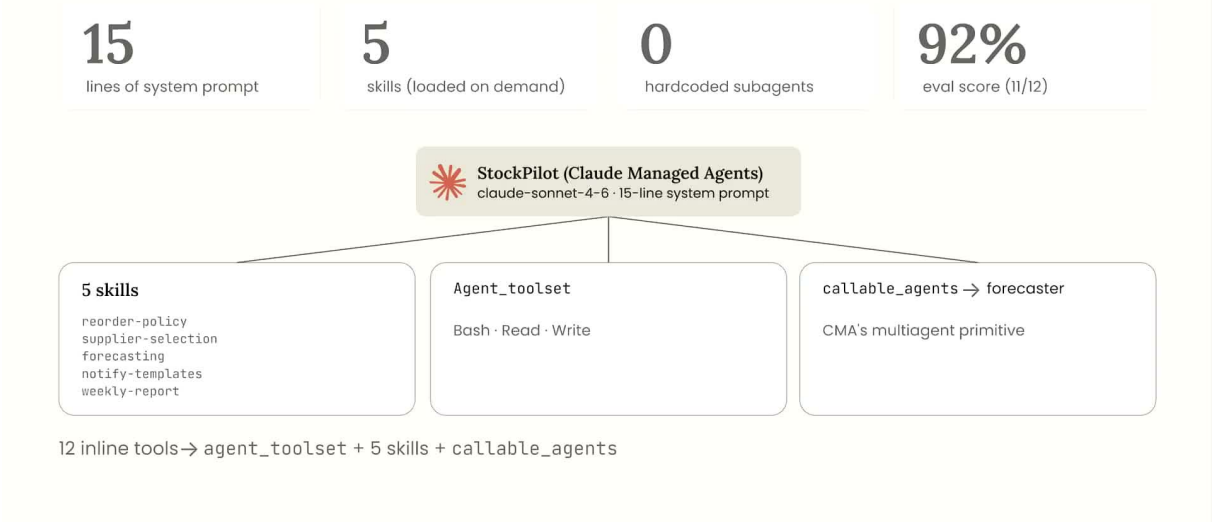


图 16: 最终架构页: 一个 CMA 上的 StockPilot agent、15 行 system prompt、5 个按需加载 skills、0 个 hardcoded subagents、约 92% eval score, 并通过 callable_agents 挂载 forecaster。¹²

7.1 最终架构: 把复杂度放回正确边界

Will 对最终结构的解释有一个清楚取舍: 基础设施交给 CMA, 因为他想关注 “building the best thing possible”, 不想把 scaling、security、hosting 和 session 管理变成 workshop 的主要变量。主 agent 仍然存在, 但职责收窄: 它是 **orchestrator**, 负责调度和整合。

架构部件	最终形态	设计含义
orchestrator	部署在 CMA 的 single agent loop	主 session 只保留身份、不可变约束和调度职责
System prompt	约 15 行	每个任务都需要的核心规则留在 prompt 中
skills	约 5 个按需加载的组织流程和业务规则包	forecast、inventory policy、报告流程等知识按任务进入上下文
tools	bash、read、write, 并同步必要数据	用代码执行和文件系统处理 CSV、预测数据和中间结果
subagent	0 个 hardcoded wrapper-style sub-agents	固定工具包装子代理被移除, 避免隐藏通信和日志
callable agents	forecaster 按条件挂载	预测需要独立上下文时, 由 CMA native path 承载

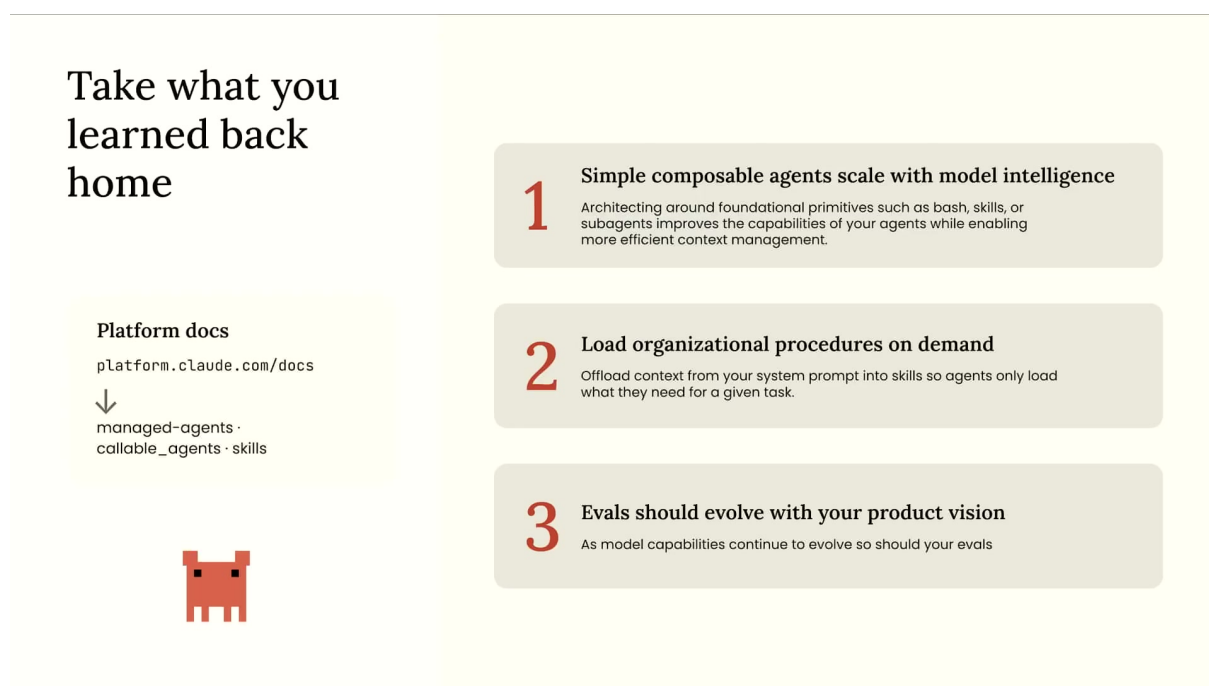
这里的 “0 个 hardcoded subagents” 需要准确理解: 它指的是不再把子代理硬编码成普通工具 wrapper。forecasting capability 仍然存在, 只是由 CMA 的 callable_agents 按条件挂载, 日志、

¹²视频画面时间区间: 00:41:00–00:42:20。

metrics 和 session 证据也随之进入托管边界。

7.2 Speaker 的收尾 takeaways

Will 的收尾讨论保留了三条实质性建议：从简单的 single agent loop 和 human-like primitives 开始；把组织流程通过 **skills** 按需加载；持续写 **evals**，并随着产品愿景扩展而更新 **evals**。他还明确给出工程取舍：当 token usage 和 cost 下降、performance 上升时，某些高智能任务允许保留略高 latency 或相近 turn count。



The slide titled "Take what you learned back home" features a light yellow background. On the left, it lists "Platform docs" with the URL "platform.claude.com/docs", followed by a downward arrow and the paths "managed-agents · callable_agents · skills". Below this is a small red robot icon. On the right, there are three numbered takeaways in grey boxes:

- 1 Simple composable agents scale with model intelligence**
Architecting around foundational primitives such as bash, skills, or subagents improves the capabilities of your agents while enabling more efficient context management.
- 2 Load organizational procedures on demand**
Offload context from your system prompt into skills so agents only load what they need for a given task.
- 3 Evals should evolve with your product vision**
As model capabilities continue to evolve so should your evals

图 17: 收尾页把 workshop 压缩为三条 takeaways: simple composable agents、on-demand procedures、evals evolve with product vision。¹³

三条 takeaways 的中文展开

第一，**simple composable agents scale with model intelligence**。围绕 foundational primitives 架构 agent，可以让新模型能力自然转化为更好的工具使用能力。

第二，**load organizational procedures on demand**。组织流程、策略、模板和业务规则进入 **skills**，让 agent 在需要时加载，减少 context pollution。

第三，**evals should evolve with product vision**。产品能力变化后，旧 eval suite 可能测不到新风险；evals 必须跟着产品目标更新。

这三条建议构成一个递进关系：先让 agent 获得足够通用的行动能力，再让它按需获得组织知识，最后用 evals 证明改动确实服务产品目标。缺少第三步时，架构改造很容易变成主观感觉；缺少前两步时，eval 改善也可能只是局部补丁。

¹³ 视频画面时间区间：00:43:13–00:44:35。

7.3 决策矩阵: tool、skill、subagent、MCP、callable agents

下面的矩阵是本讲的可复用决策框架。它把“信息应该放哪里、行动能力应该放哪里、独立认知应该放哪里”拆成五类工程选择。

选择	放什么/做什么	适用信号	主要风险	eval 观察点
tool	读写、执行、查询、调用 API、agent 需要改变外部状态或读取外部数据	运行代码等行动能力	工具过多导致选择混乱; 专门工具遮住通用能力	tool-call success、token、latency、错误重试次数
skill	流程、政策、模板、业务规则、预测步骤等按需知识	信息只在部分任务中需要, 长期塞进 prompt 会污染上下文	skill 边界过碎或命名不清, 导致加载遗漏	结构合规率、policy failure、context token 使用量
subagent	另一个拥有独立 context window 的 Claude 实例	并行探索或 fresh mind 评审能带来质量收益	通信、日志、输出契约和归因成本上升	跨 agent transcript 完整性、review hit rate、failure attribution
MCP	多 client 或多 agent 共享的受治理工具集合	工具需要标准化发布、权限治理和跨环境复用	MCP server 重叠、上下文膨胀、治理成本上升	tool discovery 准确性、权限错误、上下文占用
callable agents	CMA native managed subagents, 带 session-level observability	需要 subagent 隔离, 同时需要平台级日志、metrics 和会话追踪	平台绑定增强; 输入输出契约仍需设计	子代理 metrics、F2/R8 等任务 provenance、调用路径可追踪性

表 1: agent decomposition 决策矩阵: 按知识、行动、独立认知、治理与观测边界选择架构位置。

弱 evals 会让好架构看起来成功

Hill climbing 依赖 eval coverage。若 eval suite 没有覆盖关键业务目标、数据治理、延迟预算或新产品能力, 架构变化可能在分数上变好, 同时隐藏真实产品失败。因此, evals should evolve with your product vision 是最终 takeaways 中最容易被低估的一条。

7.4 风险与限制

第一, workshop demo 的胜利路径依赖 Stock Pilot 的任务形态。库存预测、CSV 分析和业务流程迁移很适合 bash/read/write 与 skills; 其他领域可能需要更多 custom tools、严格权限模型或专门数据管道。

第二, CMA 与 callable agents 是 Anthropic 平台能力。一般化到其他 agent 平台时, 读者应寻找等价的 session 管理、sandbox、日志、metrics 和子代理调用契约。缺少这些平台能力时, tool-wrapped subagent 的 observability 风险会重新出现。

第三, MCP 的使用条件需要单独验证。多 client、多 agent、标准化和治理需求越强, MCP 越有价值; 临时、单 agent、CLI/API 可达的能力可以先通过 local tool 或 code execution 实现。过早 MCP 化会增加上下文和治理负担。

第四, 成本改善不自动等于体验改善。Will 提到 token usage、cost 和 execution time 下降, turn count 基本相近; 在某些复杂任务里, 略高 latency 可以换来更好 performance 和更低成本。实际产品仍需用用户等待时间、失败恢复路径和人工介入率来验证。

7.5 拓展阅读

幻灯片给出的官方入口是 <https://platform.claude.com/docs>。围绕本讲主题, 优先阅读三个方向:

- **managed-agents**: 理解 CMA 如何管理 agent definition、session、sandbox 和工具环境。
- **callable_agents**: 理解 native managed subagents 如何提供更清楚的 session-level observability 与 metrics。
- **skills**: 理解 packaged and composable information 如何实现 progressive disclosure。

补充阅读时应带着同一个问题进入文档：这项能力是在放知识、放行动能力、放独立认知，还是在放治理和观测边界？这个问题能防止读者把所有新功能都塞进同一层架构。

7.6 本章小结

本讲的最终框架是：用 **evals** 定位退化，用 **skills** 承载按需知识，用 **primitive tools** 承载通用行动能力，用 MCP 承载共享且受治理的工具集合，用 **subagent** 和 **callable agents** 承载真正需要隔离上下文的认知任务。Stock Pilot 的改造证明，agent 变复杂之后的第一原则是边界复位：prompt、工具和子代理都应回到最适合的职责层，并用持续演进的 **evals** 验证边界是否有效。